

Entrées et sorties

Demander des informations et en afficher, avec input() et print()

Nicholas Wolff · CC BY-SA 4.0

Sommaire

1

Entrées et sorties

Communiquer avec la personne qui utilise le programme, et suivre dans la trace ce qui s'affiche

2

`print()`

Afficher des informations dans la console pour que la personne qui utilise le programme en prenne connaissance

3

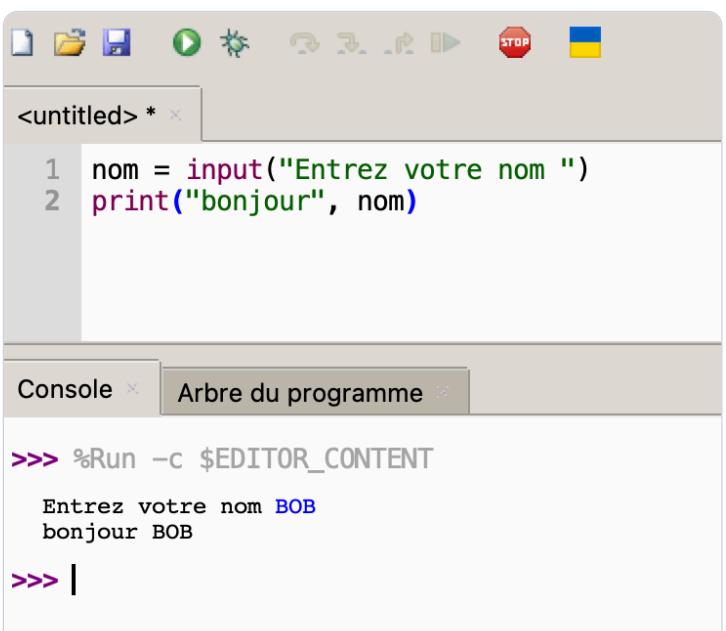
`input()`

Interrompre le programme pour demander une *valeur* à la personne qui l'utilise, et la convertir pour calculer avec

Entrée et sortie avec `input()` et `print()`

On utilise des entrées et des sorties pour communiquer avec la personne qui utilise le programme.

Quand on programme, on joue à la fois le rôle de la personne qui programme (en haut dans l'éditeur) et celui de la personne qui utilise le programme (en bas dans la console).



La ligne 1 du programme fait beaucoup de choses :

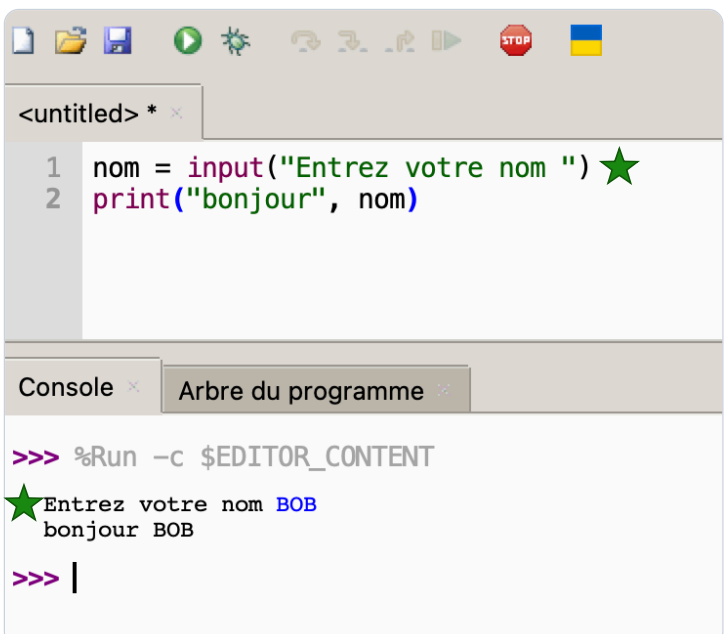
- Elle affiche la phrase en vert, le *message*, dans la console en bas
- Elle interrompt le programme
- Elle attend la réponse de la personne qui utilise le programme
- La personne saisit une réponse et appuie sur la touche Entrée
- Le programme reprend son exécution
- Il assigne la réponse donnée à la ***variable*** *nom*

La ligne 2 du programme affiche dans la console `bonjour` puis, à la suite, le *nom* saisi par la personne.

Trace quand il y a des entrées et des sorties

Quand le programme appelle `print( )`, on ajoute une colonne « affichage » à la trace.
On y met *tout* ce que le programme affiche dans la console, y compris le message de `input( )`, mais pas ce que la personne saisit : ça va dans la variable.

Pour cet exemple de programme :



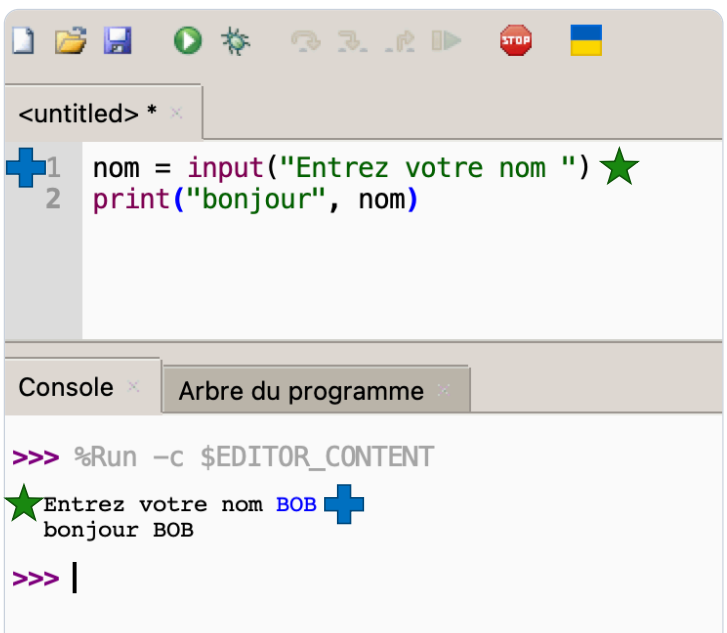
On obtient cette trace :

ligne	nom	affichage
1	"BOB"	Entrez votre nom ★
2		bonjour BOB

Trace quand il y a des entrées et des sorties

Quand le programme appelle `print( )`, on ajoute une colonne « affichage » à la trace.
On y met *tout* ce que le programme affiche dans la console, y compris le message de `input( )`, mais pas ce que la personne saisit : ça va dans la variable.

Pour cet exemple de programme :



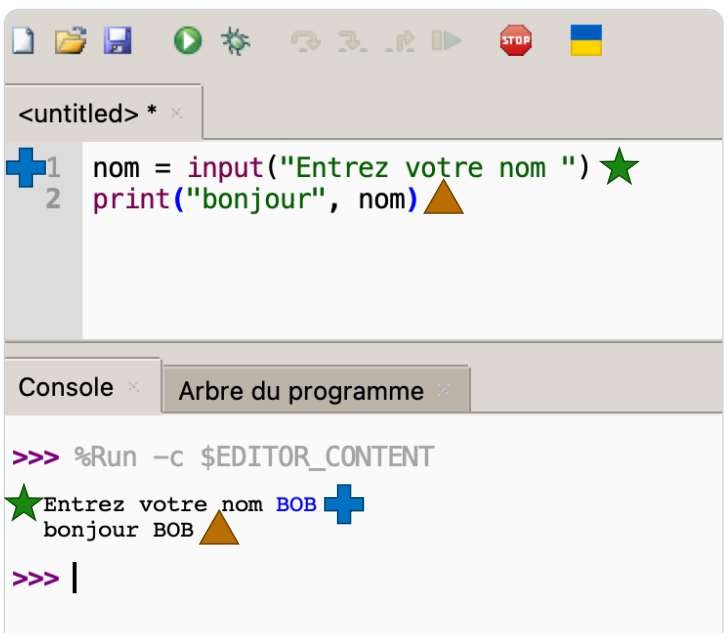
On obtient cette trace :

ligne	nom	affichage
1	"BOB"	Entrez votre nom
2		bonjour BOB

Trace quand il y a des entrées et des sorties

Quand le programme appelle `print( )`, on ajoute une colonne « affichage » à la trace.
On y met *tout* ce que le programme affiche dans la console, y compris le message de `input( )`, mais pas ce que la personne saisit : ça va dans la variable.

Pour cet exemple de programme :



On obtient cette trace :

ligne	nom	affichage
1	"BOB"	Entrez votre nom
2		bonjour BOB

Sommaire

1

Entrées et sorties

Communiquer avec la personne qui utilise le programme, et suivre dans la trace ce qui s'affiche

2

`print()`

Afficher des informations dans la console pour que la personne qui utilise le programme en prenne connaissance

3

`input()`

Interrompre le programme pour demander une *valeur* à la personne qui l'utilise, et la convertir pour calculer avec

La fonction `print()` en Python

C’est l’outil standard pour afficher des données dans la console.

Syntaxe de base :

```
print("Texte", variable, 123)
```

Affiche les éléments séparés par un espace, et ajoute un retour à la ligne automatique.

Exemples de sorties avec `print()`

Code Python	Affichage	Explication
`print("Bonjour")`	`Bonjour`	Affiche la str `"Bonjour"`
`print("print")`	`print`	Affiche la str `"print"`, sans l'interpréter
`print(4.5)`	`4.5`	Le nombre est automatiquement converti pour l'affichage
`a = 12`		
`print(a)`	`12`	Évalue a, puis affiche la valeur
`print(a + 1)`	`13`	Évalue `a + 1`, puis affiche la valeur
`print("a")`	`a`	Affiche la str `"a"`
`print("a:", a)`	`a: 12`	Affiche la str `"a:"`, puis un espace, puis la valeur de a
`print("2 *", a, "=", a * 2)`	`2 * 12 = 24`	str, puis valeur, puis str, puis expression. (Un espace s'ajoute automatiquement entre les éléments)

Concaténation ou virgules

```
prénom = "Alice"
âge = 30
```

Pour afficher la phrase complète `Bonjour Alice. Tu as 30 ans.`, on peut essayer deux façons :

A) En utilisant `print( )` avec plusieurs éléments séparés par des virgules :

```
print("Bonjour", prénom, ". Tu as", âge, "ans.")
```

```
Bonjour Alice . Tu as 30 ans.
```

B) Avec l'opération de concaténation de chaînes de caractères + :

```
print("Bonjour " + prénom + ". Tu as " + str(âge) + " ans.")
```

```
Bonjour Alice. Tu as 30 ans.
```

Comparaison :

- Avec les virgules : simple, `print( )` ajoute automatiquement des espaces entre les éléments.
- Avec + : contrôle total du texte, mais les nombres doivent être convertis avec `str( )`.

Sommaire

1

Entrées et sorties

Communiquer avec la personne qui utilise le programme, et suivre dans la trace ce qui s'affiche

2

`print()`

Afficher des informations dans la console pour que la personne qui utilise le programme en prenne connaissance

3

`input()`

Interrompre le programme pour demander une *valeur* à la personne qui l'utilise, et la convertir pour calculer avec

La fonction `input()` en Python

C’est l’outil qui permet au programme de récupérer du texte saisi au clavier par la personne qui l’utilise.

Syntaxe de base :

```
nom = input("Quel est votre nom ? ")
```

Le programme affiche le message, attend la saisie de la personne, puis assigne le résultat à la variable.

À retenir :

- Le programme s’arrête tant que la personne n’a pas appuyé sur Entrée.
- Le message entre parenthèses est optionnel, mais recommandé pour guider la personne.

Type du résultat de `input()` : conversion

```
v = input("nombre de personnes ? ")
```

La personne qui utilise le programme va saisir quelque chose au clavier et finir avec Entrée.
On assigne ce qui a été saisi à la variable v (avec v = …).

Activité · 1 min

Quel est le type de cette valeur ? Chaîne de caractères, nombre entier, nombre décimal, booléen ?

Type du résultat de `input()` : conversion


```
v = input("nombre de personnes ? ")
```

La personne qui utilise le programme va saisir quelque chose au clavier et finir avec Entrée.
On assigne ce qui a été saisi à la variable v (avec v = …).

Activité

Quel est le type de cette valeur ? Chaîne de caractères, nombre entier, nombre décimal, booléen ?

La valeur est de type **chaîne de caractères**.

En effet, il est impossible à ce stade du programme de savoir ce que la personne va saisir : on assigne donc simplement à v *exactement* les caractères qu'elle a saisis :

- par exemple `"Bob"`
- ou `""` (rien)
- ou `"11"` (qui n'est pas le nombre 11, mais une chaîne de caractères composée de deux fois le caractère `"1"`)

Type du résultat de `input()` : conversion

```
v = input("nombre de personnes ? ")
```

On assigne à la variable v la **chaîne de caractères** saisie par la personne qui utilise le programme.

Si dans le programme on veut utiliser la valeur saisie comme un nombre entier, il va falloir convertir la chaîne de caractères en nombre :

```
1 v = input("nombre de personnes ? ")
2 n = int(v)
3 print(n + 1)
```

Type du résultat de `input()` : conversion


```
v = input("nombre de personnes ? ")
```

On assigne à la variable v la **chaîne de caractères** saisie par la personne qui utilise le programme.

Si dans le programme on veut utiliser la valeur saisie comme un nombre entier, il va falloir convertir la chaîne de caractères en nombre :

```
1 v = input("nombre de personnes ? ")
2 n = int(v)
3 print(n + 1)
```

Trace dans le cas où la personne répond 4 :

ligne	v	n	affichage
1	"4"		nombre de personnes ?
2		4	
3			5

On peut aussi *combiner* `input()` et la conversion en une seule ligne :

```
1 n = int(input("nombre de personnes ? "))
2 print(n + 1)
```

`input()` est exécuté en premier, le résultat de `input()` est ensuite converti en `int`

Type du résultat de `input()` : conversion


```
v = input("nombre de personnes ? ")
```

On assigne à la variable v la **chaîne de caractères** saisie par la personne qui utilise le programme.

Si dans le programme on veut utiliser la valeur saisie comme un nombre entier, il va falloir convertir la chaîne de caractères en nombre :

```
1 v = input("nombre de personnes ? ")
2 n = int(v)
3 print(n + 1)
```

Trace dans le cas où la personne répond 4 :

ligne	v	n	affichage
1	"4"		nombre de personnes ?
2		4	
3			5

On peut aussi *combiner* `input()` et la conversion en une seule ligne :

```
1 n = int(input("nombre de personnes ? "))
2 print(n + 1)
```

Trace dans le cas où la personne répond 4 :

ligne	n	affichage
1	4	nombre de personnes ?
2		5

`input()` est exécuté en premier, le résultat de `input()` est ensuite converti en `int`

Type du résultat de `input()` : conversion

Rappel : il se peut que la conversion échoue.

```
v = input("nombre de personnes ? ")
n = int(v)
print(n + 1)
```

Le programme est correct.

Si la personne qui utilise le programme saisit un nombre, tout fonctionne bien.

Si elle saisit un nom au lieu d'un nombre comme demandé, une erreur va survenir, car il est impossible de convertir cette chaîne de caractères en nombre.

Type du résultat de `input()` : conversion

Rappel : il se peut que la conversion échoue.

```
1 v = input("nombre de personnes ? ")
2 n = int(v)
3 print(n + 1)
```

Trace dans le cas où la personne répond `Kevin` :

ligne	v	n	affichage
1	"Kevin"		nombre de personnes ?
2		Erreur	

Le programme est correct.

Si la personne qui utilise le programme saisit un nombre, tout fonctionne bien.

Si elle saisit un nom au lieu d'un nombre comme demandé, une erreur va survenir, car il est impossible de convertir cette chaîne de caractères en nombre.

Exemples d'entrées avec `input()`

Code Python

```
espèce = input()
```

```
nom = input("Quel est votre nom ? ")
```

```
âge = int(input(nom + " quel est votre âge ? "))
```

Ce qu'on voit dans la console

```

```

```
Quel est votre nom ?
```

```
Bob quel est votre âge ?
```

Sources

Contenu

Python — Fonctions natives : `input()`, `int()`, `print()`, `str()`
docs.python.org

Images

ArnoldReinhold – un téléscripteur Teletype ASR-33 (CC BY-SA 3.0)
commons.wikimedia.org

Thonny, l'éditeur Python de la capture d'écran
thonny.org